

Architektura Techniczna i Plan Wdrożenia Platformy TipJar+: Raport Wykonawczy 2025

1. Wstęp: Definicja Perfekcji w Inżynierii Oprogramowania Ery Web3

W roku 2025 krajobraz cyfrowy ewoluował w kierunku, w którym przeciętność jest synonimem porażki. Użytkownicy, przyzwyczajeni do natychmiastowej gratyfikacji i płynności interfejsów znanych z aplikacji natywnych, nie tolerują opóźnień, błędów logicznych czy niespójności wizualnych. Projekt TipJar+, mający ambicję stać się liderem w domenie Creator Economy, wymaga podejścia, które transceduje tradycyjne "dobre praktyki" programistyczne, wkraczając w sferę inżynieryjnego sztucznego. Niniejszy raport, przygotowany z perspektywy architekta systemowego o najwyższych kwalifikacjach, stanowi destylację wiedzy eksperckiej, analizy najnowszych trendów technologicznych oraz rygorystycznych standardów jakości.

Celem tego dokumentu jest nie tylko dostarczenie planu wdrożenia, ale zdefiniowanie filozofii "dopieszczonego kodu" (polished code). Kod ten charakteryzuje się nie tylko poprawnością składniową, ale także elegancją architektoniczną, przewidywalnością działania i bezkompromisową wydajnością. Analiza dokumentacji TipJar+, w połączeniu z badaniem obecnych standardów (React 19, Next.js 15, Feature-Sliced Design), prowadzi do konkluzji, że sukces platformy zależy od precyzyjnej orkiestracji trzech filarów: modularnej architektury skalowalnej, niewidocznej integracji technologii blockchain oraz interfejsu użytkownika, który zaciera granicę między fizyką a cyfrową interakcją.

W niniejszym opracowaniu, liczącym tysiące słów szczegółowej analizy, zdekonstruujemy każdy aspekt systemu. Od zarządzania stanem optymistycznym, przez bezpieczeństwo portfeli embedded, aż po niuanse typografii na ekranach mobilnych. Jest to plan dla perfekcjonisty, który rozumie, że w kodzie, tak jak w sztuce, diabeł tkwi w szczegółach.

2. Paradygmat Architektoniczny: Feature-Sliced Design (FSD) w Ekosystemie Next.js

Fundamentem każdej długowiecznej i skalowalnej aplikacji jest jej architektura. W 2025 roku, w obliczu rosnącej złożoności aplikacji frontendowych, monolityczne struktury czy proste podziały na components i pages stały się niewystarczające. Odpowiedzią na potrzebę rygorystycznej separacji odpowiedzialności jest Feature-Sliced Design (FSD), metodologia,

która wymusza dyscyplinę i przewidywalność.

2.1. Filozofia Warstw i Jednokierunkowy Przepływ Danych

Feature-Sliced Design nie jest jedynie konwencją nazewnictwa folderów; to paradygmat myślenia o systemie jako o zestawie funkcjonalności biznesowych, a nie technicznych.¹ Kluczową zasadą, którą TipJar+ musi zaadoptować, jest ścisła hierarchia warstw. Zależności mogą przebiegać wyłącznie "w dół". Warstwa wyższa może korzystać z niższej, ale nigdy odwrotnie. To zapobiega powstawaniu cyklicznych zależności, które są plagą dużych projektów Reactowych.

W kontekście TipJar+, struktura ta będzie wyglądać następująco, od warstwy najwyższej do najniższej:

1. **App (app/)**: Warstwa inicjalizacyjna. To tutaj następuje kompozycja całej aplikacji. Zawiera globalne style, konfigurację providerów (Theme, Web3, Query Client) oraz, w przypadku Next.js, definicje routingu. Jest to jedyna warstwa, która ma prawo "widzieć" wszystkie pozostałe.
2. **Processes (processes/)**: (Opcjonalnie) Warstwa orkiestracji skomplikowanych procesów biznesowych, które angażują wiele stron lub kroków, np. wieloetapowy proces KYC (Know Your Customer) lub onboarding nowego twórcy. W 2025 roku tendencją jest przenoszenie tej logiki do Features lub Widgets, ale w przypadku TipJar+ może być niezbędna do zarządzania stanem między widokami.²
3. **Pages (pages/)**: Warstwa kompozycji widoków. Strona w FSD nie powinna zawierać "twardej" logiki biznesowej. Jej zadaniem jest ułożenie Widgetów i Feature'ów w spójną całość dla konkretnego adresu URL. Na przykład CreatorProfilePage importuje DonationWidget i CreatorBioEntity.
4. **Widgets (widgets/)**: Samodzielne, duże bloki UI. Widgety łączą w sobie logikę z warstw niższych (features, entities) w gotowe do użycia sekcje, np. Header, TransactionHistoryTable czy NFTGallery. Widget jest kompletnym elementem interfejsu, który można wstawić na stronę.
5. **Features (features/)**: Serce logiki biznesowej z perspektywy użytkownika. To tutaj definiujemy "co użytkownik może zrobić". Przykłady dla TipJar+: SendTip, ConnectWallet, MintBadge. Feature zawiera kod UI (np. przycisk), model (stan, hooki) oraz integrację z API.
6. **Entities (entities/)**: Reprezentacja bytów biznesowych. Tutaj znajdują się definicje typów danych (User, Transaction, NFT), proste komponenty wyświetlające te dane (np. UserAvatar, TransactionRow) oraz logika pobierania danych (queries). Encje są "pasywne" – nie zawierają logiki interakcji biznesowych.¹
7. **Shared (shared/)**: Fundament techniczny. Biblioteki UI (przyciski, inputy – tzw. UI Kit), narzędzia (formatowanie dat, walut), konfiguracja API, stałe. Warstwa ta musi być absolutnie niezależna od logiki biznesowej aplikacji.

2.2. Rozwiązanie Konfliktu Strukturalnego: Next.js App Router a FSD

Implementacja FSD w Next.js 13+ (i nowszym 15) napotyka na fundamentalny problem: Next.js wymusza strukturę opartą na systemie plików w katalogu app dla routing, co stoi w sprzeczności z płaską, domenową strukturą FSD.³ Bezpośrednie wrzucanie warstw FSD do folderu app Next.js prowadzi do chaosu, gdzie logika miesza się z routingiem.

Ekspertcka analiza³ sugeruje rozwiązanie hybrydowe, które rekomenduję dla TipJar+. Polega ono na traktowaniu folderu app Next.js wyłącznie jako warstwy infrastrukturalnej routing, podczas gdy właściwa implementacja stron znajduje się w src/pages (zgodnie z FSD).

Proponowana, perfekcyjna struktura katalogów:

```
src/
├── app/                # Next.js App Router (tylko routing i proxy)
│   ├── layout.tsx     # Globalny layout (Providers)
│   ├── page.tsx       # Home Page (importuje z @/pages/home)
│   └── /
│       ├── page.tsx   # Profil twórcy (importuje z @/pages/creator-profile)
│       └── api/       # Route Handlers (proxy do logiki w features/entities)
├── pages/             # Warstwa FSD: Pages
│   ├── home/         # Slice strony głównej
│   │   ├── ui/       # Komponenty strony
│   │   └── index.ts  # Public API
│   ├── creator-profile/ # Slice profilu twórcy
├── widgets/           # Warstwa FSD: Widgets
│   ├── navigation/   # Navbar, Footer
│   ├── donation-panel/ # Panel wpłat (złożony z features)
│   └── activity-feed/ # Feed aktywności (złożony z entities)
├── features/          # Warstwa FSD: Features (Interakcje)
│   ├── send-tip/     # Logika wysyłania napiwku
│   │   ├── ui/       # Formularz, Modal
│   │   ├── model/    # useSendTip, walidacja Zod
│   │   └── api/      # Server Actions (integracja z blockchain)
│   ├── wallet-auth/  # Logika logowania portfelem
│   └── claim-reward/ # Odbieranie nagród NFT
├── entities/          # Warstwa FSD: Entities (Dane)
│   ├── session/      # Aktualna sesja użytkownika
│   └── creator/       # Model danych twórcy
```

└─ transaction/	# Model transakcji
└─ shared/	# Warstwa FSD: Shared
└─ ui/	# Design System (Radix UI + Tailwind)
└─ api/	# Instancje klienta (Axios/Fetch/Wagmi)
└─ lib/	# Utils (cn, formatters)
└─ config/	# Zmienne środowiskowe

Taka struktura gwarantuje, że zmiany w routingu Next.js nie wymuszają refaktoryzacji logiki biznesowej, a logika biznesowa jest w pełni przenośna i testowalna w izolacji. Jest to podejście "kuloodporne", charakteryzujące się wysoką kohezją (cohesion) i niskim sprzężeniem (coupling).

2.3. Publiczne API i Enkapsulacja Modułów

Dla programisty o wysokim IQ, kod jest formą komunikacji. Aby ta komunikacja była jasna, FSD wprowadza pojęcie Public API dla każdego slice'a. Każdy folder w features, entities itd. musi zawierać plik index.ts, który eksportuje **wyłącznie** to, co jest przeznaczone do użytku zewnętrznego.⁶

Wnętrznosci modułu (np. pomocnicze hooki, lokalne stałe, sub-komponenty) powinny pozostać ukryte. To jest klucz do "dopieszczonego kodu" – minimalizacja powierzchni ataku na zmiany. Jeśli zmieniamy wewnętrzną implementację features/send-tip, a Public API pozostaje bez zmian, mamy gwarancję, że reszta aplikacji (np. widgets/donation-panel) nadal działa poprawnie. Narzędzia takie jak ESLint z pluginem eslint-plugin-boundaries lub eslint-plugin-import powinny być skonfigurowane tak, aby fizycznie uniemożliwić "głębokie importy" (deep imports) z pominięciem index.ts.⁷

3. Fundamenty Technologiczne: React 19 i Nowa Era Interaktywności

TipJar+, jako projekt z horyzontem wdrożenia na rok 2025, musi w pełni adoptować nowości wprowadzone w React 19. Nie jest to kwestia "bycia na czasie", ale fundamentalnej poprawy User Experience i Developer Experience.

3.1. Server Components (RSC) a Architektura Hybrydowa

W erze React 19, domyślnym sposobem renderowania są Server Components. Dla TipJar+ oznacza to, że większość kodu UI (layouty, strony profilowe, listy NFT) będzie renderowana na serwerze, co drastycznie redukuje ilość JavaScriptu przesyłanego do klienta (tzw. hydration cost).⁸

Komponenty oznaczone dyrektywą "use client" będą stosowane chirurgicznie – tylko tam,

gdzie wymagana jest interaktywność (np. obsługa kliknięć, zarządzanie stanem formularza, animacje Framer Motion). Strategia ta wymaga zmiany myślenia: zamiast pobierać dane w `useEffect` po stronie klienta, pobieramy je bezpośrednio w komponencie serwerowym (często w warstwie Pages lub Widgets w strukturze FSD) i przekazujemy jako propsy do interaktywnych wysp (Client Components).

3.2. Optimistic UI: Eliminacja Percepcji Opóźnień

W aplikacjach finansowych i Web3 opóźnienia sieciowe są nieuniknione (czas bloku, finalizacja transakcji). Dla perfekjonisty, UI musi być jednak "szybsze niż myśl". React 19 wprowadza hook `useOptimistic`, który jest rewolucją w tym zakresie.⁹

Implementacja w TipJar+ (Feature send-tip):

Zamiast czekać na potwierdzenie transakcji z blockchaina (co może trwać 10-30 sekund), interfejs musi zareagować natychmiast.

1. Użytkownik klika "Wyślij Napiwek".
2. `useOptimistic` natychmiast aktualizuje stan lokalny: dodaje wpis do historii transakcji i zwiększa licznik wsparcia na profilu twórcy. Użytkownik widzi efekt w < 50ms.
3. W tle uruchamiana jest Server Action (`sendMessageAction`), która komunikuje się z siecią.
4. Jeśli akcja się powiedzie, React automatycznie revaliduje dane i zastępuje stan optymistyczny rzeczywistym (który powinien być już zgodny).
5. Jeśli wystąpi błąd, React automatycznie wycofuje zmianę (rollback) do poprzedniego stanu, a my wyświetlamy powiadomienie o błędzie (Toast).

Kluczowe jest tu wykorzystanie API `startTransition`, które pozwala Reactowi priorytetyzować te aktualizacje i nie blokować głównego wątku. To jest "krok po kroku" do płynności natywnej aplikacji.¹⁰

3.3. Server Actions jako Nowe API

TipJar+ odchodzi od tworzenia oddzielnych endpointów API w `pages/api` na rzecz Server Actions. Pozwalają one na wywoływanie funkcji backendowych (np. zapisu do bazy danych, interakcji z RPC węzła) bezpośrednio z event handlerów w komponentach. Dzięki temu logika biznesowa (np. walidacja, autoryzacja) znajduje się tuż obok kodu UI, który ją wywołuje, co zwiększa kohezję kodu i ułatwia typowanie (pełne wsparcie TypeScript bez konieczności generowania typów dla REST/GraphQL).⁸

4. Web3 Integration Strategy: Niewidoczna Technologia

Dla użytkownika końcowego ("non-crypto native"), technologia blockchain powinna być niewidoczna. Wymaganie instalacji wtyczki MetaMask i zarządzania seed phrase w 2025 roku

jest błędem projektowym.¹²

4.1. Analiza Porównawcza Embedded Wallets: Privy vs. Dynamic

Rynek rozwiązań "Wallet-as-a-Service" (WaaS) został zdominowany przez graczy oferujących embedded wallets. Analiza dostępnych rozwiązań ¹⁴ wskazuje na konieczność wyboru między Privy a Dynamic.

Kryterium	Privy	Dynamic	Implikacje dla TipJar+
Architektura Bezpieczeństwa	MPC (Multi-Party Computation) - klucz dzielony	MPC / Account Abstraction	MPC jest niezbędne dla bezpieczeństwa non-custodial przy zachowaniu odzyskiwalności konta.
UX Onboardingu	Focus na konwersję (Email/SMS -> Wallet)	Focus na gotowe, piękne UI widgety	Privy oferuje "niewidzialność" portfela, co jest kluczowe dla masowego odbiorcy.
Customizacja	Headless-first (pełna kontrola nad UI)	Pre-built components (trudniejsze do pełnego brandingu)	Privy wygrywa dla "perfekcjonisty", który chce mieć kontrolę nad każdym pikselem formularza logowania.
Koszt (Skala)	MAU-based (droższy przy dużej skali)	MAU-based (tańszy start)	Koszt jest drugorzędny wobec jakości UX i konwersji w modelu premium.

Rekomendacja: Wybór **Privy** w trybie headless. Pozwala to na stworzenie całkowicie customowego flow logowania (zgodnego z Design Systemem TipJar+), gdzie użytkownik

loguje się e-mailem, a w tle tworzony jest bezpieczny portfel embedded.

4.2. Progressive Onboarding & KYC

Zgodnie z trendami compliance na 2025 ¹⁶, TipJar+ wdroży strategię progresywnego onboarding.

1. **Faza 1 (Wspierający):** Rejestracja samym e-mailem/social (Google/Apple). Tworzony jest portfel embedded. Brak KYC. Użytkownik może wpłacać środki (on-ramp) i tipować.
2. **Faza 2 (Twórca - Start):** Rejestracja e-mail. Możliwość przyjmowania wpłat, ale środki są zablokowane do wypłaty.
3. **Faza 3 (Twórca - Wypłata):** Przy pierwszej próbie wypłaty lub przekroczeniu progu AML (np. 1000 EUR), wymagane jest pełne KYC (weryfikacja dokumentu).

Taka struktura minimalizuje tarcie (friction) na wejściu, maksymalizując konwersję rejestracji, jednocześnie spełniając wymogi regulacyjne UE/Global.¹⁶

4.3. Zarządzanie Zasobami Cyfrowymi (NFT/IPFS)

TipJar+ prawdopodobnie będzie obsługiwać NFT jako "odznaki" dla wspierających. Ładowanie obrazów z publicznych bramek IPFS (ipfs.io) jest w 2025 roku nieakceptowalne ze względu na wydajność.¹⁸

Strategia Content Delivery:

- Wykorzystanie **dedykowanej bramki IPFS** (np. poprzez Pinata lub Filebase).
- **Image Optimization:** Dynamiczna transformacja obrazów na poziomie bramki (np. `?width=500&format=webp`).²⁰
- **Caching:** Metadane NFT i linki do zoptymalizowanych obrazów będą cache'owane w bazie danych TipJar+ (np. Postgres + Redis) podczas indeksowania, aby UI nigdy nie czekało na odpowiedź sieci IPFS.

5. Inżynieria Interfejsu: UX, Design i Dostępność

"Perfekcyjnie wykończone elementy" wymagają podejścia inżynierskiego do designu. Nie chodzi tylko o estetykę, ale o matematyczną precyzję, fizykę ruchu i uniwersalną dostępność.

5.1. Mobile-First i Precyzja Tailwind CSS

W 2025 roku projektowanie "desktop-first" jest błędem. Interfejs musi być skalowalny od smartfona po ekrany 4K.

- **4-Point Grid:** System odstępów oparty na wielokrotności 4px. W Tailwind: p-1 (4px), m-4 (16px), gap-6 (24px). Konsekwencja w stosowaniu tego systemu buduje podświadome poczucie harmonii u użytkownika.²¹

- **Touch Targets:** Bezwzględne przestrzeganie minimalnego obszaru interaktywnego 44x44px. Nawet jeśli ikona ma 16px, jej padding lub niewidzialny obszar kliknięcia musi spełniać ten wymóg. Jest to krytyczne dla użyteczności mobilnej.²¹
- **Fluid Typography:** Użycie clamp() dla rozmiarów czcionek, aby tekst płynnie skalował się między breakpointami, zamiast skokowych zmian.

5.2. Fizyka Animacji: Framer Motion

Animacje w TipJar+ nie są ozdobnikiem, lecz informacją zwrotną. Wykorzystamy **Layout Projection** z Framer Motion, aby stworzyć interfejsy, które płynnie zmieniają swój układ zamiast skokowego przerysowania.²³

Zastosowanie Layout Projection:

Gdy użytkownik rozwija kartę transakcji, zamiast animować height (co powoduje reflow i jest kosztowne dla CPU), używamy propa layout. Framer Motion oblicza pozycję początkową i końcową, a następnie aplikuje transformację scale i translate na warstwie kompozycji GPU.

- **Problem:** Skalowanie kontenera powoduje zniekształcenie dzieci (np. tekst staje się rozciągnięty).
- **Rozwiązanie:** Zastosowanie "Scale Correction" – nadanie propa layout również elementom potomnym (tekst, obrazki). Framer Motion automatycznie aplikuje kontr-skalę, dzięki czemu element powiększa się, ale jego zawartość pozostaje ostra i niezniekształcona.²⁶
- **Fizyka:** Użycie sprężyn (type: "spring") zamiast krzywych Beziera dla interakcji fizycznych, co daje odczucie "ciężaru" i "bezwładności" elementów UI.

5.3. Dostępność (WCAG 2.1 AA) jako Fundament

Dostępność nie jest opcją, jest wymogiem profesjonalizmu.

- **Focus Trap:** Wszelkie modale i panele boczne muszą "więzić" fokus klawiatury. Użytkownik nie może wyjść tabulatorem poza otwarty modal do tła strony.²⁹
- **Kontrast Kolorów:** Stosunek kontrastu tekstu do tła musi wynosić min. 4.5:1. Dla kolorów marki (np. złoty #FFD700), tekst musi być czarny (#000000), a nie biały, co potwierdzają obliczenia kontrastu (14.97:1 dla czerni vs 1.40:1 dla bieli – co jest niedopuszczalne).³⁰
- **Semantyka:** Używanie aria-live="polite" dla dynamicznych zmian w UI (np. pojawienie się nowego napiwku w feedzie), aby czytniki ekranu informowały użytkownika o zmianie bez przerywania mu.³³

6. Inżynieria Wydajności: Wirtualizacja i Zarządzanie Danymi

Przy dużej skali danych (tysiące transakcji), standardowe renderowanie Reacta jest

niewystarczające.

6.1. Wirtualizacja List: React Virtuoso

Analiza porównawcza bibliotek do wirtualizacji ³⁴ wskazuje jednoznacznie na **React Virtuoso** jako następcę React Window/Virtualized w 2025 roku.

- **Przewaga:** React Virtuoso obsługuje elementy o *zmiennej i nieznanej wysokości* bez konieczności ręcznego pomiaru. Jest to kluczowe dla feedu aktywności TipJar+, gdzie każdy wpis może mieć inną długość tekstu.
- **Mechanizm:** Virtuoso renderuje tylko te elementy, które są w viewporcie (+ bufor), automatycznie obserwując zmiany rozmiaru DOM i dostosowując pasek przewijania. Pozwala to na wyświetlanie list o długości 100k+ elementów przy stałym użyciu pamięci i 60fps przy scrollowaniu.

6.2. Internacjonalizacja i Formatowanie Walut

Obsługa wielu walut (USD, EUR, ETH, SOL) wymaga precyzji.

- **Rozwiązanie:** Użycie natywnego API Intl.NumberFormat w połączeniu z lekkim wrapperem lub biblioteką react-intl.³⁸ Pozwala to na formatowanie krypto z odpowiednią liczbą miejsc po przecinku i poprawnymi separatorami dla danej lokalizacji użytkownika, bez narzutu dużych bibliotek typu numeral.js.

7. Plan Wdrożenia: Mapa Drogowa do Perfekcji

Realizacja tak ambitnego projektu wymaga podziału na precyzyjne etapy. Poniższy plan zakłada metodykę iteracyjną, gdzie każdy krok kończy się dostarczeniem "perfekcyjnie wykonanego elementu".

Faza 1: Fundament Architektoniczny (Tygodnie 1-2)

- **Inicjalizacja:** Setup repozytorium Next.js 15 + TypeScript. Konfiguracja CI/CD (GitHub Actions) z rygorystycznymi regułami lintera (zakaz any, wymuszone sortowanie importów).
- **Struktura FSD:** Utworzenie szkieletu folderów `src/{app,pages,widgets,features,entities,shared}`. Konfiguracja aliasów ścieżek (`@/features`, `@/shared`) w `tsconfig.json`.
- **Design System:** Implementacja tokenów w Tailwind CSS (kolory, typografia, spacing). Stworzenie biblioteki podstawowych komponentów (Atomów) w `shared/ui` przy użyciu Radix UI (primitives) dla zapewnienia dostępności.
- **Dark Mode:** Implementacja przełączania motywów bez efektu migotania (FOUC).

Faza 2: Tożsamość i Web3 Core (Tygodnie 3-4)

- **Privy Integration:** Wdrożenie Privy SDK w trybie headless. Stylowanie formularzy logowania (e-mail/social).
- **Entity Session:** Zarządzanie sesją użytkownika i stanem portfela embedded.
- **Wallet Features:** Implementacja podstawowych funkcji portfela (wyświetlanie adresu, salda) w warstwie features/wallet.
- **Security:** Konfiguracja weryfikacji podpisów (SIWE - Sign-In with Ethereum) po stronie serwera dla autoryzacji akcji.

Faza 3: Transakcje i Optimistic UI (Tygodnie 5-7)

- **Feature SendTip:** Budowa formularza wpłaty. Implementacja hooka useOptimistic do natychmiastowej aktualizacji UI po kliknięciu "Wyślij".
- **Server Actions:** Implementacja logiki biznesowej po stronie serwera (zapis do DB, interakcja z blockchainem/procesorem płatności).
- **Widget TransactionFeed:** Implementacja wirtualizowanej listy wpłat przy użyciu React Virtuoso.
- **Obsługa Błędów:** Implementacja mechanizmów rollback i powiadomień Toast w przypadku niepowodzenia transakcji łańcuchowej.

Faza 4: Zasoby Cyfrowe i Profil Twórcy (Tygodnie 8-9)

- **Page CreatorProfile:** Kompozycja widżetów na stronie profilowej.
- **NFT Integration:** Integracja z bramką IPFS (Pinata/Filebase). Implementacja komponentu NFTImage z automatycznym doбором jakości i formatu.
- **Feature MintReward:** Logika przyznawania NFT za wsparcie.

Faza 5: Polerowanie i Audyty (Tygodnie 10-12)

- **A11y Audit:** Pełny przegląd z narzędziami automatycznymi (Axe) i manualne testy klawiaturą. Weryfikacja kontrastów.
- **Performance Tuning:** Analiza bundle'a, optymalizacja obrazów, weryfikacja Core Web Vitals (LCP, CLS, INP).
- **Animation Polish:** Dodanie mikro-interakcji i przejść layoutu (Framer Motion).
- **Security Audit:** Przegląd kodu pod kątem bezpieczeństwa smart kontraktów i API.

8. Podsumowanie

Przedstawiony plan wdrożenia platformy TipJar+ to manifest technicznego perfekcjonizmu. Wykorzystując najnowocześniejsze, ale stabilne technologie (React 19, Next.js 15, FSD, Privy), tworzymy system, który jest nie tylko funkcjonalny, ale także piękny w swojej strukturze i działaniu. Jest to podejście bezkompromisowe, wymagające wysokich kompetencji, ale gwarantujące rezultat na światowym poziomie – aplikację, która w 2025 roku wyznaczy standardy dla całej branży Web3 Creator Economy. Każdy element kodu, każda interakcja i każda decyzja architektoniczna służą tu jednemu celowi: dostarczeniu użytkownikowi

doświadczenia absolutnie płynnego, bezpiecznego i satysfakcjonującego.

Cytowane prace

1. Feature-Sliced Design Architecture in React with TypeScript: A Comprehensive Guide | by Codewithzahid | Oct, 2025 | Medium, otwierano: grudnia 24, 2025, <https://medium.com/@codewithxohii/feature-sliced-design-architecture-in-react-with-typescript-a-comprehensive-guide-b2652283c6b2>
2. Frontend Masters: Feature-Sliced Design (FSD) Pattern | by ismail harmanda | Stackademic, otwierano: grudnia 24, 2025, <https://blog.stackademic.com/frontend-masters-feature-sliced-design-fsd-pattern-81416088b006>
3. Usage with Next.js | Feature-Sliced Design, otwierano: grudnia 24, 2025, <https://feature-sliced.design/docs/guides/tech/with-nextjs>
4. Feature Sliced Design in Next JS. What is FSD and why is it needed ? | by Sriramanvellingiri, otwierano: grudnia 24, 2025, <https://medium.com/@sriramanvellingiri/feature-sliced-design-in-next-js-7d20be4338de>
5. Best Practices for Organizing Your Next.js 15 2025 - DEV Community, otwierano: grudnia 24, 2025, <https://dev.to/bajrayejeon/best-practices-for-organizing-your-nextjs-15-2025-53ji>
6. Tutorial | Feature-Sliced Design - GitHub Pages, otwierano: grudnia 24, 2025, <https://feature-sliced.github.io/documentation/docs/get-started/tutorial>
7. Mastering Feature-Sliced Design: Lessons from Real Projects - DEV Community, otwierano: grudnia 24, 2025, <https://dev.to/arjunsanthosh/mastering-feature-sliced-design-lessons-from-real-projects-2ida>
8. Mastering React 19 Part 2: Server Components & Server Actions [Tutorial] - Scalable Path, otwierano: grudnia 24, 2025, <https://www.scalablepath.com/react/react-19-server-components-server-actions>
9. How to Use the Optimistic UI Pattern with the useOptimistic() Hook in React - freeCodeCamp, otwierano: grudnia 24, 2025, <https://www.freecodecamp.org/news/how-to-use-the-optimistic-ui-pattern-with-the-useoptimistic-hook-in-react/>
10. useOptimistic - React, otwierano: grudnia 24, 2025, <https://react.dev/reference/react/useOptimistic>
11. React 19 useOptimistic Hook Breakdown - DEV Community, otwierano: grudnia 24, 2025, <https://dev.to/dthompsondev/react-19-useoptimistic-hook-breakdown-5g9k>
12. Blockchain UX: How to Design User-Friendly Decentralized Apps - Purrrweb, otwierano: grudnia 24, 2025, <https://www.purrrweb.com/blog/blockchain-ux-design/>
13. Web3 UX Design Patterns that Build Trust - 10 Interface Decisions - Coinbound,

- otwierano: grudnia 24, 2025,
<https://coinbound.io/web3-ux-design-patterns-that-build-trust/>
14. Top 7 Privy Alternatives in 2025 - Openfort, otwierano: grudnia 24, 2025,
<https://www.openfort.io/blog/privy-alternatives>
 15. Choosing the Right Embedded Wallet for Your Web3 Application - Gelato, otwierano: grudnia 24, 2025,
<https://gelato.cloud/blog/choosing-the-right-embedded-wallet-for-your-web3-application>
 16. Top 7 KYC & AML Trends That Will Shape Compliance in 2025, otwierano: grudnia 24, 2025,
<https://kyc-chain.com/top-7-kyc-aml-trends-that-will-shape-compliance-in-2025/>
 17. Fintech KYC Trends & Best Practices: The Future of KYC in 2026 - AU10TIX, otwierano: grudnia 24, 2025,
<https://www.au10tix.com/blog/fintech-trends-and-best-practices/>
 18. Enhancing NFT Marketplaces with IPFS Gateways - Firebase, otwierano: grudnia 24, 2025,
<https://firebase.com/blog/enhancing-nft-marketplaces-with-ipfs-gateways/>
 19. Why Every NFT Marketplace Uses the Same 3 IPFS Gateways (And What Happens When They Fail) | by Sohail Saifi | Medium, otwierano: grudnia 24, 2025,
https://medium.com/@sohail_saifi/why-every-nft-marketplace-uses-the-same-3-ipfs-gateways-and-what-happens-when-they-fail-4c96ebe3dde8
 20. How to Optimize Images with IPFS Dedicated Gateways - Pinata, otwierano: grudnia 24, 2025,
<https://pinata.cloud/blog/how-to-optimize-images-with-ipfs-dedicated-gateway/>
 21. Mobile Website Design Best Practices for 2025: A Complete Guide - Webstacks, otwierano: grudnia 24, 2025,
<https://www.webstacks.com/blog/mobile-website-design-best-practices>
 22. Mobile-First Web Design: Best Practices for 2025, otwierano: grudnia 24, 2025,
<https://www.alfdesigngroup.com/post/best-practices-for-mobile-first-websites>
 23. layout-projection.md - GitHub Gist, otwierano: grudnia 24, 2025,
<https://gist.github.com/taowen/e102cf5731e527cb9ac02574783c4119>
 24. Everything about Framer Motion layout animations - The Blog of Maxime Heckel, otwierano: grudnia 24, 2025,
<https://blog.maximeheckel.com/posts/framer-motion-layout-animations/>
 25. Framer motion - Layout Animations - DEV Community, otwierano: grudnia 24, 2025,
<https://dev.to/siddharth0x/framer-motion-layout-animations-50kh>
 26. Layout Animations - Scale correction for child elements - Fixed - Framer Motion examples, otwierano: grudnia 24, 2025,
<https://framermotionexamples.com/example/layout-animations-scale-correction-for-child-elements-fixed>
 27. Layout Animation — React FLIP & Shared Element - Motion, otwierano: grudnia 24, 2025,
<https://motion.dev/docs/react-layout-animations>
 28. Layout Animations » Scale Correction for Child Elements - The Framer Code

- Guide, otwierano: grudnia 24, 2025,
<https://framer.mighty.guide/layout-animations/scale-correction-for-child-elements/>
29. React Focus Trap Accessibility - CoreUI, otwierano: grudnia 24, 2025,
<https://coreui.io/react/docs/components/focus-trap/accessibility/>
 30. Color #FFD700 — deep, warm - Color Combos, otwierano: grudnia 24, 2025,
<https://www.colorcombos.com/colors/ffd700>
 31. #ffd700 Color Hex Gold1, otwierano: grudnia 24, 2025,
<https://www.color-hex.com/color/ffd700>
 32. #eab308 Color Hex, otwierano: grudnia 24, 2025,
<https://www.color-hex.com/color/eab308>
 33. Web Content Accessibility Guidelines (WCAG): Guide (2025) - Parallel HQ,
otwierano: grudnia 24, 2025,
<https://www.parallelhq.com/blog/what-are-web-content-accessibility-guidelines-wcag>
 34. Optimizing Large Datasets with Virtualized Lists | by Eva Matova | Medium,
otwierano: grudnia 24, 2025,
<https://medium.com/@eva.matova6/optimizing-large-datasets-with-virtualized-lists-70920e10da54>
 35. react-virtualized vs. react-window - LogRocket Blog, otwierano: grudnia 24, 2025,
<https://blog.logrocket.com/react-virtualized-vs-react-window/>
 36. Infinite Scrolling Made Easy: react-window vs react-virtuso | by Stuthi Neal -
Medium, otwierano: grudnia 24, 2025,
<https://medium.com/@stuthineal/infinite-scrolling-made-easy-react-window-vs-react-virtuso-1fd786058a73>
 37. React Virtualization - react-window vs react-virtuoso - DEV Community,
otwierano: grudnia 24, 2025,
<https://dev.to/sanamumtaz/react-virtualization-react-window-vs-react-virtuoso-8g>
 38. Core FormatJS Intl | Format.JS, otwierano: grudnia 24, 2025,
<https://formatjs.github.io/docs/intl/>
 39. Simplify Currency Formatting in React: A Zero-Dependency Solution with Intl API,
otwierano: grudnia 24, 2025,
<https://dev.to/josephciullo/simplify-currency-formatting-in-react-a-zero-dependency-solution-with-intl-api-3kok>